LABORATORIO 4

DISEÑO DE CÓDIGO

MELISA GUZMÁN MUÑOZ

DISEÑO DE DATOS Y ALGORITMOS

EDWARD ALEXANDER FRANCIA GUZMAN

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

BOGOTÁ, 2026

Descripción:

Se obtienen n estudiantes de los cuales hay m notas (el valor de m ya no es fijo para todos los estudiantes), se requiere calcular el promedio de cada uno de los estudiantes, decir si aprobó o no y finalmente decir el promedio general del curso

Requerimientos:

RF01: El sistema debe permitir ingresar la cantidad de estudiantes.

RF02: El sistema debe permitir ingresar el nombre del estudiante y una cantidad variable de notas.

RF03: El sistema debe validar que cada estudiante tenga al menos una nota ingresada.

RF04: El sistema debe calcular el promedio individual de cada estudiante según la cantidad de notas ingresadas.

RF05: El sistema debe mostrar:

- Nombre del estudiante
- Lista de notas
- Promedio individual

RF06: El sistema debe indicar si el estudiante aprobó o no, considerando que aprueba con promedio mayor o igual a 3.0.

RF07: El sistema debe calcular el promedio general del curso.

RF08: El sistema debe usar una estructura de programación dinámica para calcular el promedio general del curso mediante acumulación progresiva.

Historias de usuario:

H01: Permitir ingresar la cantidad de estudiantes del curso.

H02: Permitir ingresar el nombre y una cantidad variable de notas por cada estudiante.

H03: Validar que cada estudiante tenga al menos una nota ingresada.

H04: Calcular el promedio individual de cada estudiante según la cantidad de notas ingresadas.

H05: Mostrar el nombre, las notas y el promedio individual de cada estudiante.

H06: Determinar si el estudiante aprueba o no con base en un promedio mínimo de 3.0.

H07: Calcular el promedio general del curso utilizando programación dinámica.

Diagramas:

Proceso previo:

1. Ingresar cantidad de estudiantes.
2. Ingresar nombre y notas de cada estudiante.
3. Calcular promedio individual.
4. Determinar si aprobó.
5. Guardar promedios en una lista.
6. Aplicar programación dinámica para calcular el promedio general.
7. Mostrar resultados.

Diagrama de flujo:

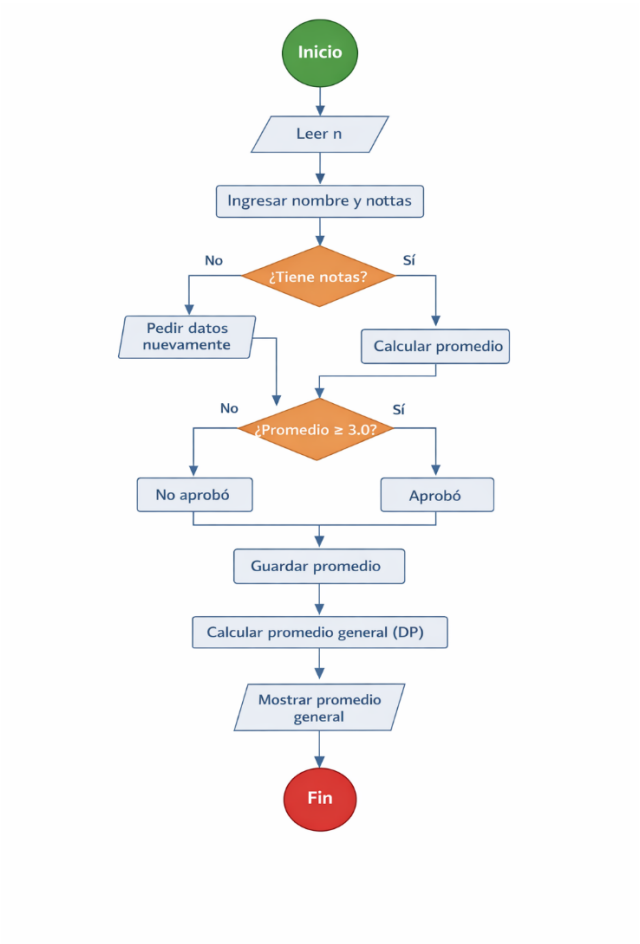


Diagrama de secuencia

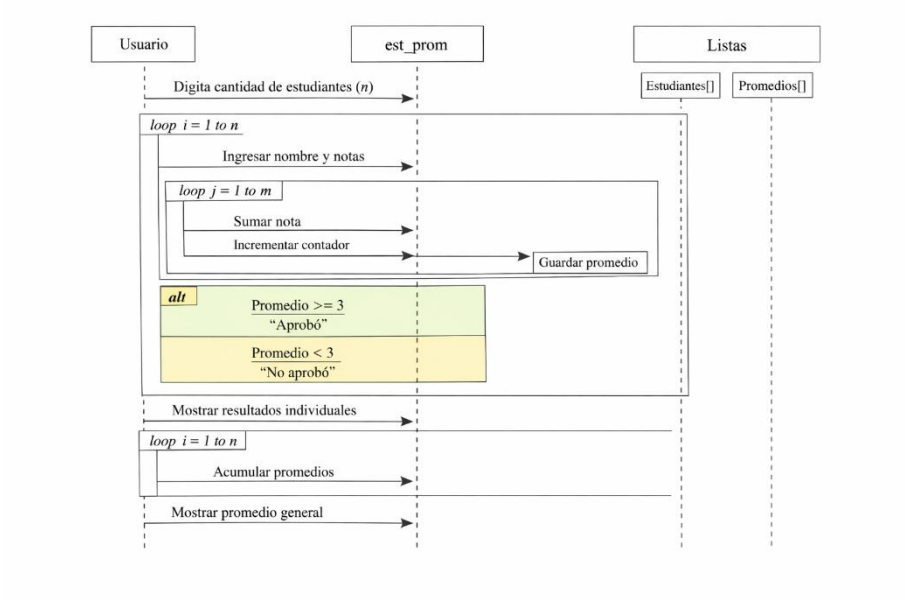
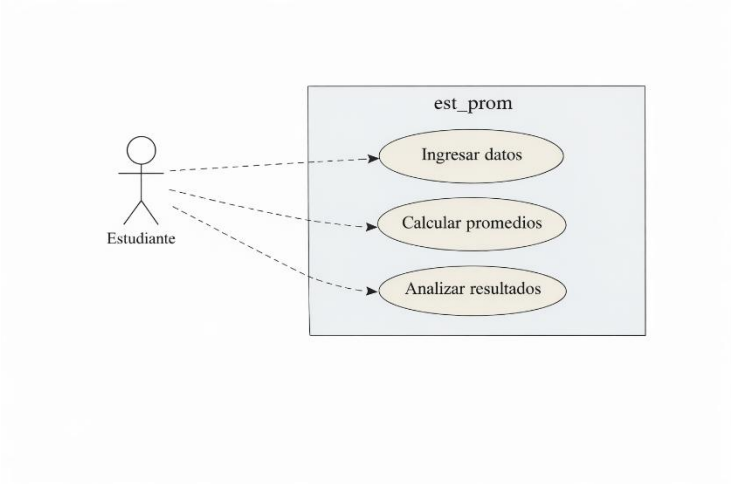


Diagrama casos de uso:



Análisis de complejidad:

Línea / Instrucción	**Costo**	**Veces que se ejecuta**	**Total**
n = int(input())	c1	1	c1
promedios = []	c2	1	c2
estudiantes = []	c3	1	c3
for i in range(n)	c4	n	c4n
input().split()	c5	n	c5n

Línea / Instrucción	Costo	Veces que se ejecuta	Total
for j in range(1,len(datos))	c6	n-1(m)	c6(n-1)(m)
suma = suma + nota	c7	(n-1)(m-1)	C7(n-1)(m-1)
cantidad = cantidad + 1	c8	(n-1)(m-1)	C9 (n-1)(m-1)
notas.append(nota)	c9	(n-1)(m-1)	C8(n-1)(m-1)
promedios.append(promedio)	c10	n	c10n
estudiantes.append(...)	c11	n	c11n
for estudiante in estudiantes	c12	n	c12n
for i in range(n) (promedio general)	c13	n	c13n

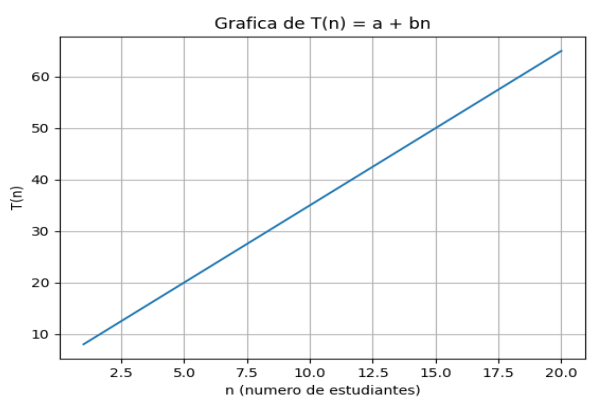
Ecuación $T(n) = c_1 + c_2 + c_3 + (c_4 + c_5 + c_{10} + c_{11} + c_{12} + c_{13})n + c_6(n-1)m + (c_7 + c_8 + c_9)(n-1)(m-1)$

$$T(n) = a\,nm + b\,n + c$$

Mejor caso: $T(n) = c_1 + c_2 + c_3 + (c_4 + c_5 + c_{10} + c_{11} + c_{12} + c_{13})n + c_6(n-1)$

$$T(n) = a + bn$$

Gráfica mejor caso:



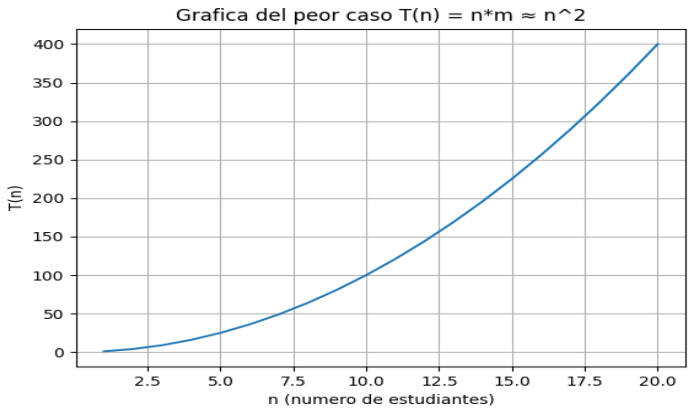
Peor caso:

$$T(n) = c_1 + c_2 + c_3 + (c_4 + c_5 + c_{10} + c_{11} + c_{12} + c_{13})n + c_6(nm - m) + (c_7 + c_8 + c_9)(nm - n - m + 1)$$

$$T(n) = a\,nm + b\,n + d\,m + e$$

$$T(n) = O(nm)$$

Gráfica peor caso:



Análisis propio respecto a la complejidad:

El algoritmo presenta una complejidad que depende del número de estudiantes (n) y de la cantidad de notas por estudiante (m), ya que existe un ciclo que recorre a los estudiantes y dentro de este otro que procesa sus notas; debido a esta estructura anidada, la complejidad general es (O(nm)). En el mejor caso, cuando cada estudiante tiene solo una nota (m = 1), el crecimiento es lineal (O(n)), mientras que en el peor caso, si el número de notas crece proporcionalmente con (n), la complejidad se aproxima a (O(n^2)). Además, el uso de un arreglo acumulador para calcular el promedio general refleja una aplicación básica de programación dinámica, ya que se aprovechan resultados parciales para obtener el resultado final de manera organizada.

Test:

El objetivo de las pruebas es verificar que el programa calcule correctamente los promedios individuales, determine si el estudiante aprueba o no (promedio ≥ 3.0), calcule el promedio general del curso y maneje correctamente casos límite.

Caso de prueba 1 – Caso normal

Entrada:

2

Ana 4.0 3.0 5.0

Luis 2.0 3.0

Resultado esperado:

Ana → Promedio 4.0 → Aprobó

Luis → Promedio 2.5 → No aprobó

Promedio general del curso: 3.25

Caso de prueba 2 – Mejor caso (una nota por estudiante)

Entrada:

3

Carlos 4.0

Maria 3.5

Sofia 2.8

Resultado esperado:

Carlos → 4.0 → Aprobó

Maria → 3.5 → Aprobó

Sofia → 2.8 → No aprobó

Promedio general calculado correctamente.

Caso de prueba 3 – Caso límite

Entrada:

0

Resultado esperado:

No hay estudiantes.

Código con los tests:

```python
def est_prom_testable(datos_estudiantes):
    promedios = []

    for datos in datos_estudiantes:
        nombre = datos[0]
        notas = datos[1]

        suma = 0
        cantidad = 0

        for nota in notas:
            suma = suma + nota
            cantidad = cantidad + 1

        promedio = suma / cantidad
        promedios.append(promedio)

    suma_total = 0
    for p in promedios:
        suma_total = suma_total + p

    promedio_general = suma_total / len(promedios)

    return promedios, promedio_general
# Caso 1
datos1 = [
    ("Ana", [4.0, 3.0, 5.0]),
    ("Luis", [2.0, 3.0])
]

proms, pg = est_prom_testable(datos1)

assert round(proms[0], 2) == 4.00
assert round(proms[1], 2) == 2.50
assert round(pg, 2) == 3.25


# Caso 2 – Una sola nota por estudiante
datos2 = [
    ("Carlos", [4.0]),
    ("Maria", [3.5]),
    ("Sofia", [2.8])
]

proms, pg = est_prom_testable(datos2)

assert round(proms[0], 2) == 4.00
assert round(proms[1], 2) == 3.50
assert round(proms[2], 2) == 2.80


print("Todos los tests pasaron correctamente.")
```

Código entregable:

```python
def est_prom():
    n = int(input("Digite la cantidad de estudiantes: "))

    promedios = []
    estudiantes = []

    for i in range(n):
        datos = input("Ingrese nombre y sus notas (puede ser cantidad diferente para cada estudiante) separadas por espacio: ").split()

        while len(datos) < 2:
            datos = input("Error, digite nuevamente: ").split()

        nombre = datos[0]
        suma = 0.0
        cantidad = 0
        notas = []

        for j in range(1, len(datos)):
            nota = float(datos[j])
            suma = suma + nota
            cantidad = cantidad + 1
            notas.append(nota)

        promedio = suma / cantidad
        promedios.append(promedio)
        estudiantes.append((nombre, notas, promedio))

    print()

    for estudiante in estudiantes:
        nombre = estudiante[0]
        notas = estudiante[1]
        promedio = estudiante[2]

        print("Nombre:", nombre)
        print("Notas:", notas)
        print("Promedio:", promedio)

        if promedio >= 3.0:
            print("Aprobó")
        else:
            print("No aprobó")
        print()

    dp = []
    for i in range(n + 1):
        dp.append(0)

    for i in range(1, n + 1):
        dp[i] = dp[i - 1] + promedios[i - 1]

    promedio_general = dp[n] / n

    print("Promedio general del curso:", promedio_general)

est_prom()
```